KIMI3 昇腾 A3 算子开发
设计 · 验证 · 测试报告

目标设备：昇腾 A3（Ascend 910_9382 · SOC ascend910_93） | CANN 9.0.0 | torch_npu 2.10.0

交付范围：华为 CANN 尚未覆盖的 KIMI3 算子 · AscendC 实现 + 参考实现 + 验证 harness

1. 概述

1.1 背景

KIMI3 是 MoonshotAI 的混合架构大模型（MLA / KDA 线性注意力 + MoE）。在昇腾 NPU 上部署时，需要把模型每一层映射到 CANN 算子。经与华为 CANN 现有算子库比对，KIMI3 特有算子存在两类缺口：① 华为完全没实现的算子（需从零写 AscendC kernel）；② 华为有但形状/接口不匹配、实为图接线问题的算子。

1.2 交付物清单

类别	内容
自研 AscendC 算子	SiTU-GLU（SituAndMul）、AttnRes 残差栈
参考实现（golden）	reference/situ.py、attn_res.py、kda.py、kda_chunked.py
torch 包装	situ_and_mul_wrapper.cpp、attn_res_wrapper.cpp
验证 harness	test_situ_real.py、test_attn_res_real.py、compare_real_chunk_kda.py
文档	本报告 + README.md + docs/（环境、开发流程、KDA 对比）

2. 缺口分析

算子	华为覆盖	结论
MLA attention	ATB MultiLatentAttention	√ 已有
MoE dispatch/combine	npu_ops_transformer	√ 已有
sigmoid 路由 + topk	ATB FusedAddTopkDiv	√ 已有
KDA（Gated Delta Rule）	aclnnChunkKdaFwd	√ 已有（补 wrapper + 验证）
SiTU-GLU	× 只有 SiLU	自研 AscendC kernel
AttnRes 残差栈	× 无	自研 AscendC kernel
LatentMoE 投影	× 形状不符	图级组合（无需新 kernel）

结论：华为真正缺的算子只有 2 个（SiTU-GLU、AttnRes），本仓库均已实现并验证；LatentMoE 是模型图接线问题，不属算子开发。

3. 算子设计

3.1 SiTU-GLU（SituAndMul）

数学定义（取自 modeling_kimi_linear.py 的 SituAndMul）：输入 $x \in [..., 2d]$（前 d 为 gate、后 d 为 up），输出 $y \in [..., d]$。

```
gate = x[..., :d] ; up = x[..., d:]
situ(gate) = beta·tanh(gate/beta)·sigmoid(gate) ( beta=4.0 )
up' = linear_beta·tanh(up/linear_beta) ( linear_beta=25.0 )
y = situ(gate) · up'
```

与 SiLU（x·sigmoid(x)）不同：SiTU 是 tanh·sigmoid 乘积，且 up 分支带 tanh 压缩，是 KIMI3 特有激活变体，华为只有 SiLU 融合。

接口：aclnnSituAndMul(x, beta, linearBeta, out)，x/out 为 bf16，out 形状 [..., d]。

Kernel 设计：沿最后一维劈两半（gate 前 d、up 后 d）；d 维按 TILE_ELEM=4096 分块；内部 fp32 计算；输入 DataCopy(MTE2)→向量(V) 与 向量(V)→输出 DataCopy(MTE3) 用 TQue 的 EnQue/DeQue 同步；按 token 行分 8 core。

3.2 AttnRes 残差栈

数学定义（取自 _apply_attn_res）：本质是「mini-attention」，对 B+1 个残差向量做 RMS-norm + 学习 score 点积 + softmax + 加权求和。

```
v = cat([block_residual, prefix_sum[:,None]], dim=1) (T, B+1, H)
k = v · rsqrt(mean(v², -1) + eps)
scores = sum(k · score_weight, -1)
probs = softmax(scores, -1)
out = sum(probs[:,:,None] · v, 1)
```

其中 score_weight = norm.weight · proj.weight.squeeze(0) ∈ (H,) 为学到的常量。

接口：aclnnAttnRes(blockResidual, prefixSum, scoreWeight, eps, out)，blockResidual (T,B,H)、prefixSum (T,H)、scoreWeight (H,)、out (T,H)，均 bf16。

Kernel 设计：两趟——Pass 1 逐残差向量沿 H 归约 sum(v²) 与 sum(v·score_weight)，得 score 后向量 Exp 在 32 对齐 buffer 上做 softmax；Pass 2 加权求和 out = Σ_b probs[b]·v[b]。归约用 ReduceSum(dst, src, workBuf, H) + GetValue(0)。按 token 分 8 core。

4. 验证方法论

无权重验证：只靠 shape + 随机数据 + 参考实现验证算子正确性，无需下载模型权重。

- 从 KIMI3 config 拿 shape/dtype；
- 固定 seed 生成随机输入；
- 参考实现（纯 torch，fp32 内部 → bf16 输出）跑出 golden；
- 待验算子（AscendC kernel，NPU 上）跑候选输出；
- 对比：shape/dtype 一致 + 数值误差在容差内（bf16 用 max_rel < 1e-2）。

为什么 golden 可信：golden 是纯 torch 的直白实现，与 NPU kernel 完全解耦、不共享任何实现，作为正确性金标准。

5. 测试结果

5.1 SiTU-GLU

shape	d	max_abs	max_rel	结果
(2, 3, 128)	64	0.0	0.0	√
(4, 5, 256)	128	0.0	0.0	√
(2, 7, 4096)	2048	9.5e-7	8.3e-4	√
(2, 7, 22016)	11008	1.56e-2	7.5e-3	√
(3, 5, 256)	128	0.0	0.0	√

（d=11008 为 KIMI3 真实 intermediate_size；末行用不同 beta/linear_beta 验证参数化。）

5.2 AttnRes

T	B	H	max_abs	max_rel	结果
4	2	512	9.5e-7	8.8e-4	√
8	4	4096	9.8e-4	4.7e-3	√
16	8	4096	1.56e-2	7.7e-3	√
32	4	1024	3.9e-3	7.4e-3	√
2	1	512	0.0	0.0	√

（H=4096 为 KIMI3 真实 hidden_size。）

5.3 KDA（华为算子，仅验证）：真实华为 chunk_kda_fwd vs 参考实现，attn_out / final_state 相对误差 ~4e-3（详见 docs/03）。

6. 关键实现要点与坑

- bf16 类型是 bfloat16_t（DT_BF16 是宏=27，不是类型）；
- REGISTER_TILING_DEFAULT 须在 GET_TILING_DATA 之前（它声明 tilingData）；
- 跨引擎同步必须 EnQue/DeQue（漏了 → 输出全零，最易踩的坑）；
- 标量 exp 在 aicore 上不存在，softmax 用向量 AscendC::Exp；
- uint→float cast 不允许，倒数类量移到 host 端算好传进 tiling；
- 归约用 ReduceSum(dst, src, workBuf, count) + GetValue(0)。

完整流程见 docs/02-AscendC算子开发流程与坑.md。

7. 结论与后续工作

结论：华为缺的 2 个真算子（SiTU-GLU、AttnRes）均已实现 AscendC kernel 并在 A3 上验证通过，误差在 bf16 精度内。LatentMoE 不需要新 kernel——它是图级组合：down_proj(Linear H→moe_hidden) → MoE → [RMSNorm] → up_proj(Linear moe_hidden→H)，全用现有算子拼，属 vllm/sglang 适配层的图接线。

后续工作（建议由昇腾团队或部署侧完成）：① LatentMoE 图接线（vllm-ascend/sglang 插入 down/up 投影）；② 性能 benchmark（本报告只验证正确性，未测吞吐/显存收益）；③ 算子合入 CANN（按 docs/02 工程结构对齐规范）。

8. 附录

8.1 环境信息

项	值
设备	Ascend 910_9382 × 2（SOC ascend910_93）
CANN	9.0.0（/usr/local/Ascend/cann-9.0.0）
torch / torch_npu	2.10.0+cpu / 2.10.0
AscendC 编译器	ccec / bisheng
算子脚手架	msopgen

8.2 复现步骤

```
# 1. 编译算子（以 SiTU-GLU 为例） cd kernels/SituAndMul && bash build.sh # 产出 build_out/custom_opp_ubuntu_aarch64.run # 2. 安装到 vendor cd build_out && ./custom_opp_ubuntu_aarch64.run --install-path=/usr/local/Ascend/cann-9.0.0/opp --quiet # 3. 跑对拍（自动 JIT 编译 wrapper + golden 对比） cd /workspace/kimi3_ops && python test_situ_real.py # AttnRes 同理：python test_attn_res_real.py
```